1. import numpy as np

```
# Part (a)
arr = np.array([[1, 2, 3], [4, 5, 6]])

print("2D Array:")
print(arr)
print("Shape of the array:", arr.shape)
print("Number of dimensions:", arr.ndim)
print("Total number of elements:", arr.size)
print("Data type of elements:", arr.dtype)
print("Transpose of the array:")
print(arr.T)

# Part (b)
zeros_array = np.zeros((2, 3), dtype=int)
ones_array = np.ones((3, 3), dtype=int)
identity_matrix = np.eye(3, dtype=int)
range_array = np.arange(0, 11, 2)
linspace_array = np.linspace(0, 1, 4)
concatenated_array = np.concatenate((np.array([1, 2, 3]), np.array([4, 5, 6])))

print("\n2x3 array of zeros:")
print(zeros_array)
print("\n3x3 array of ones:")
print(ones_array)
print("\n3x3 identity matrix:")
print(identity_matrix)
print("\nArray from 0 to 10 with step 2:")
print(range_array)
print("\n4 equally spaced numbers between 0 and 1:")
```

```
print(linspace_array)
print("\nConcatenated array:")
print(concatenated_array)
```

2. import numpy as np

```
arr1d = np.array([10, 20, 30, 40, 50, 60])
print("Original 1D array:", arr1d)

arr2d = arr1d.reshape(2, 3)
print("\n2D array of shape (2,3):")
print(arr2d)

arr3d = arr1d.reshape(2, 1, 3)
print("\n3D array of shape (2,1,3):")
print(arr3d)

flattened = arr3d.flatten()
raveled = arr3d.ravel()
print("\nFlattened array:", flattened)
print("Raveled array:", raveled)

print("\nSecond row from 2D array:", arr2d[1])
print("First column from 2D array:", arr2d[:, 0])
print("Element at position [0,0,1] in 3D array:", arr3d[0, 0, 1])

print("\nSum:", np.sum(arr2d))
print("Mean:", np.mean(arr2d))
print("Maximum:", np.max(arr2d))
print("Minimum:", np.min(arr2d))
print("Standard deviation:", np.std(arr2d))
```

3. import numpy as np

```
import matplotlib.pyplot as plt

X = np.array([1, 2, 3, 4, 5, 6, 7, 8])
Y = np.array([2, 4, 6, 8, 11, 12, 14, 16])

slope, intercept = np.polyfit(X, Y, 1)
predicted_Y = slope * X + intercept

print("Slope:", slope)
print("Intercept:", intercept)
print("Predicted Y values:", predicted_Y)

plt.scatter(X, Y, color='blue', label='Original Data Points')
plt.plot(X, predicted_Y, color='red', label='Regression Line')
plt.xlabel('X Values')
plt.ylabel('Y Values')
plt.title('Linear Regression using NumPy and Matplotlib')
plt.legend()
plt.grid(True)
plt.show()
```

4. import pandas as pd

```
# Part (a)
roll_series = pd.Series(
   [101, 102, 103, 104, 105],
   index=['Aarav', 'Diya', 'Rohan', 'Ananya', 'Vihaan']
)
print("Pandas Series:")
print(roll_series)

# Part (b)
data = {
   'Name': ['Aarav', 'Diya', 'Rohan', 'Ananya', 'Vihaan', 'Kavya', 'Aditya'],
   'Age': [20, 21, 20, 19, 22, 20, 21],
   'City': ['Mumbai', 'Delhi', 'Bangalore', 'Chennai', 'Pune', 'Kolkata', 'Hyderabad'],
   'Marks': [85, 90, 78, 92, 88, 84, 91],
   'Course': ['B.Tech', 'B.Sc', 'B.Com', 'B.Tech', 'BCA', 'B.Tech', 'B.Sc']
}

students_df = pd.DataFrame(data)
print("\nComplete DataFrame:")
print(students_df)

print("\nFirst 3 rows:")
print(students_df.head(3))

print("\nLast 2 rows:")
print(students_df.tail(2))

print("\nB.Tech students:")
print(students_df[students_df['Course'] == 'B.Tech'])
```

```
students_df.to_csv('students.csv', index=False)
read_df = pd.read_csv('students.csv')
print("\nData read back from students.csv:")
print(read_df)
```

5. import pandas as pd

```
employee_data = {
   'Employee': ['E001', 'E002', 'E003', 'E004', 'E005', 'E006', 'E007'],
   'Salary': [35000, 48000, 52000, 29000, 61000, 40000, 55000],
   'Experience': [2, 4, 5, 1, 7, 3, 6]
}

emp_df = pd.DataFrame(employee_data)

print("First 3 rows:")
print(emp_df.head(3))
print("\nLast 2 rows:")
print(emp_df.tail(2))
print("\nShape:", emp_df.shape)
print("\nInfo:")
emp_df.info()
print("\nDescribe:")
print(emp_df.describe())

print("\nSalary statistics:")
print(emp_df['Salary'].describe()[['count', 'mean', 'std', 'min', '25%', '50%', '75%', 'max']])
```

5.b) import pandas as pd

```
player_data = {
   'Player': ['Rohit', 'Kohli', 'Rahul', 'Hardik', 'Jadeja', 'Dhoni', 'Pant'],
   'Runs': [45, 72, 28, 55, 35, 40, 60],
   'Balls': [32, 48, 25, 30, 28, 22, 38]
}
```

```
player_df = pd.DataFrame(player_data)


print("First 3 rows:")
print(player_df.head(3))
print("\nLast 2 rows:")
print(player_df.tail(2))
print("\nShape:", player_df.shape)
print("\nInfo:")
player_df.info()
print("\nDescribe:")
print(player_df.describe())

print("\nRuns statistics:")
print(player_df['Runs'].describe()[['count', 'mean', 'std', 'min', '25%', '50%', '75%', 'max']])
```

6. import pandas as pd

```
students_data = {
    'Name': ['Aarav', 'Diya', 'Rohan', 'Ananya', 'Vihaan', 'Kavya', 'Aditya'],
    'Age': [20, 21, 20, 19, 22, 20, 21],
    'City': ['Mumbai', 'Delhi', 'Bangalore', 'Chennai', 'Pune', 'Kolkata', 'Hyderabad'],
    'Marks': [85, 90, 78, 92, 88, 84, 91],
    'Course': ['B.Tech', 'B.Sc', 'B.Com', 'B.Tech', 'BCA', 'B.Tech', 'B.Sc']
}

students = pd.DataFrame(students_data)
print("Original DataFrame:")
print(students)

print("\nFirst row using iloc:")
print(students.iloc[0])

print("\nFirst 3 rows using iloc:")
print(students.iloc[:3])

print("\nFirst 2 rows and first 3 columns using iloc:")
print(students.iloc[:2, :3])

print("\nLast 2 rows using iloc:")
print(students.iloc[-2:])

print("\nAll rows and last 2 columns using iloc:")
print(students.iloc[:, -2:])

students.iloc[3, 2] = 'Delhi'
print("\nDataFrame after updating position (3, 2) to 'Delhi':")
```

```
print(students)


print("\nAlternate rows (0,2,4,6) using iloc:")
print(students.iloc[[0, 2, 4, 6]])
```

7. import pandas as pd

```
import numpy as np

employees = pd.DataFrame({
   'Employee': ['Aarav', 'Diya', 'Rohan', 'Ananya', 'Vihaan', 'Kavya', 'Aditya'],
   'Salary': [50000, 60000, np.nan, 55000, np.nan, 48000, 62000],
   'Bonus': [5000, np.nan, 3000, 4500, np.nan, 4000, np.nan],
   'City': ['Mumbai', 'Delhi', np.nan, 'Chennai', 'Pune', 'Kolkata', np.nan],
   'Experience': [5, 6, 4, np.nan, 3, 5, 7]
})

print("Original DataFrame:")
print(employees)

print("\nMissing values using isnull():")
print(employees.isnull())

print("\nNon-missing values using notnull():")
print(employees.notnull())

missing_per_column = employees.isnull().sum()
total_missing = employees.isnull().sum().sum()
print("\nMissing values per column:")
print(missing_per_column)
print("Total missing values:", total_missing)

cleaned_df = employees.copy()
cleaned_df['Salary'].fillna(cleaned_df['Salary'].mean(), inplace=True)
cleaned_df['Bonus'].fillna(0, inplace=True)
cleaned_df['City'].fillna('Unknown', inplace=True)
cleaned_df['Experience'].fillna(cleaned_df['Experience'].median(), inplace=True)
```

```
cleaned_df['City'] = cleaned_df['City'].replace('Unknown', 'Not Specified')

print("\nCleaned DataFrame after filling and replacing values:")
print(cleaned_df)

rows_dropped = employees.dropna()
columns_dropped = employees.dropna(axis=1)

print("\nRows after dropping any missing value:")
print(rows_dropped)

print("\nColumns after dropping any missing value:")
print(columns_dropped)

print("\nMissing value count before cleaning:")
print(employees.isnull().sum())
print("Total before cleaning:", employees.isnull().sum().sum())

print("\nMissing value count after cleaning:")
print(cleaned_df.isnull().sum())
print("Total after cleaning:", cleaned_df.isnull().sum().sum())
```

8. #include <iostream>

```
#include <string>
using namespace std;

class Student {
private:
    int rollNo;
    float marks;

protected:
    float attendance;

public:
    string name;
    string course;

    void getData() {
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Course: ";
        cin >> course;
        cout << "Enter Roll No: ";
        cin >> rollNo;
        cout << "Enter Marks: ";
        cin >> marks;
        cout << "Enter Attendance (%): ";
        cin >> attendance;
    }

    void display() {
        cout << "Name: " << name << endl;
```

```
        cout << "Course: " << course << endl;
        cout << "Roll No: " << rollNo << endl;
        cout << "Marks: " << marks << endl;
        cout << "Attendance: " << attendance << "%" << endl;
        cout << "--------------------------" << endl;
    }

    float getMarks() {
        return marks;
    }

    float getAttendance() {
        return attendance;
    }
};

int main() {
    Student s[5];
    float totalMarks = 0;
    int highestIndex = 0, lowestIndex = 0;

    cout << "Enter details for 5 students:\n";
    for (int i = 0; i < 5; i++) {
        cout << "\nStudent " << i + 1 << endl;
        s[i].getData();
    }

    cout << "\nAll Student Details:\n";
    for (int i = 0; i < 5; i++) {
        s[i].display();
        totalMarks += s[i].getMarks();
```

```
    if (s[i].getMarks() > s[highestIndex].getMarks()) {
        highestIndex = i;
    }
    if (s[i].getMarks() < s[lowestIndex].getMarks()) {
        lowestIndex = i;
    }
}

cout << "\nHighest Marks Student:\n";
s[highestIndex].display();

cout << "\nLowest Marks Student:\n";
s[lowestIndex].display();

cout << "\nAverage Marks: " << totalMarks / 5 << endl;

cout << "\nStudents with attendance > 75%:\n";
for (int i = 0; i < 5; i++) {
    if (s[i].getAttendance() > 75) {
        s[i].display();
    }
}

cout << "\nDemonstrating access specifiers:\n";
cout << "Public members can be accessed directly:\n";
cout << "Name: " << s[0].name << endl;
cout << "Course: " << s[0].course << endl;

cout << "\nPrivate and protected members cannot be accessed directly from main().\n";
cout << "The following lines would cause errors if uncommented:\n";
```

```
    // cout << s[0].rollNo << endl;
    // cout << s[0].attendance << endl;


    return 0;
}
```

9 a).

```
#include <iostream>
#include <string>
using namespace std;

class Employee {
protected:
    int empId;
    string name;
    float basicSalary;

public:
    void getEmployeeData() {
        cout << "Enter Employee ID: ";
        cin >> empId;
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Basic Salary: ";
        cin >> basicSalary;
    }

    void showEmployeeData() {
        cout << "Employee ID: " << empId << endl;
        cout << "Name: " << name << endl;
        cout << "Basic Salary: " << basicSalary << endl;
    }
};

class Bonus : public Employee {
private:
    float bonus;
```

```
public:
    void getBonus() {
        cout << "Enter Bonus Amount: ";
        cin >> bonus;
    }

    void calculateTotalSalary() {
        showEmployeeData();
        cout << "Bonus: " << bonus << endl;
        cout << "Total Salary: " << basicSalary + bonus << endl;
    }
};

int main() {
    Bonus b;
    b.getEmployeeData();
    b.getBonus();

    cout << "\nEmployee Salary Details:\n";
    b.calculateTotalSalary();

    return 0;
}
```

9 b)

```
#include <iostream>
```

```
using namespace std;

class Animal {
public:
   void eat() {
      cout << "Animal can eat" << endl;
   }
};

class Dog : public Animal {
public:
   void bark() {
      cout << "Dog can bark" << endl;
   }
};

class Puppy : public Dog {
public:
   void weep() {
      cout << "Puppy can weep" << endl;
   }
};

int main() {
   Puppy p;
   p.eat();
   p.bark();
   p.weep();

   return 0;
}
```

10 a)

```
#include <iostream>
using namespace std;

class Father {
public:
    void house() {
        cout << "Father's House" << endl;
    }
};

class Mother {
public:
    void jewelry() {
        cout << "Mother's Jewelry" << endl;
    }
};

class Child : public Father, public Mother {
public:
    void display() {
        cout << "Child inherits from both parents" << endl;
    }
};

int main() {
    Child c;
    c.house();
    c.jewelry();
    c.display();
```

```
    return 0;
}
```

10 b)

```
#include <iostream>
using namespace std;

class Person {
public:
    void showPerson() {
        cout << "This is Person Class" << endl;
    }
};

class Student : public Person {
public:
    void showStudent() {
        cout << "This is Student Class" << endl;
    }
};

class Teacher : public Person {
public:
    void showTeacher() {
        cout << "This is Teacher Class" << endl;
    }
};
```

```
int main() {
    Student s;
    Teacher t;

    s.showPerson();
    s.showStudent();

    t.showPerson();
    t.showTeacher();

    return 0;
}
```

11.

```
#include <iostream>
using namespace std;

class Calculator {
public:
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
};

class Animal {
public:
    virtual void sound() {
        cout << "Animal makes a sound" << endl;
    }
};

class Dog : public Animal {
public:
    void sound() override {
        cout << "Dog barks" << endl;
    }
```

```
};

class Cat : public Animal {
public:
    void sound() override {
        cout << "Cat meows" << endl;
    }
};

int main() {
    Calculator calc;
    cout << "Addition of two integers: " << calc.add(10, 20) << endl;
    cout << "Addition of three integers: " << calc.add(10, 20, 30) << endl;
    cout << "Addition of two doubles: " << calc.add(10.5, 20.7) << endl;

    Animal* a;
    Dog d;
    Cat c;

    a = &d;
    a->sound();

    a = &c;
    a->sound();

    return 0;
}
```

12.

```
#include <iostream>
#include <fstream>
#include <stdexcept>
using namespace std;

int main() {
   try {
      int numerator, denominator;
      cout << "Enter numerator: ";
      cin >> numerator;
      cout << "Enter denominator: ";
      cin >> denominator;

      if (cin.fail()) {
         throw invalid_argument("Invalid input entered.");
      }

      if (denominator == 0) {
         throw runtime_error("Division by zero is not allowed.");
      }

      cout << "Result of division: " << numerator / denominator << endl;

      int arr[5]
```